



MODUL 12 · AUTHENTICATION & AUTHORIZATION

Protected Routes & proxy.ts (Next.js 16)



proxy.ts sebagai gating awal. Tapi ingat: ini BUKAN satu-satunya layer security!

BETTER AUTH & SECURITY

proxy.ts — Gating Awal di Network Level

Menggantikan middleware.ts di Next.js 16. Berjalan di Node.js runtime. Redirect user yang belum login.

```
import { NextRequest, NextResponse } from 'next/server'
import { betterFetch } from '@better-fetch/fetch'
import type { Session } from '@lib/auth'

// Route yang perlu dilindungi
const PROTECTED = ['/dashboard', '/admin', '/profile', '/settings']
const AUTH_ROUTES = ['/login', '/register']

export async function proxy(req: NextRequest) {
  const { pathname } = req.nextUrl
  const isProtected = PROTECTED.some((p) => pathname.startsWith(p))
  const isAuthRoute = AUTH_ROUTES.some((p) =>
    pathname.startsWith(p))

  // Ambil session dari Better Auth API
  const { data: session } =
    (await betterFetch)(<
    Session>
    )('/api/auth/get-session', {
      {
        baseURL: req.nextUrl.origin,
        headers: { cookie: req.headers.get('cookie') ?? '' },
      }
    })

  const isLoggedIn = !!session?.user

  // Redirect ke login jika belum login dan akses protected route
  if (isProtected && !isLoggedIn) {
    const loginUrl = new URL('/login', req.url)
    loginUrl.searchParams.set('callbackUrl', pathname)
    return NextResponse.redirect(loginUrl)
  }

  // Redirect ke dashboard jika sudah login dan akses auth route
  if (isAuthRoute && isLoggedIn) {
    return NextResponse.redirect(new URL('/dashboard', req.url))
  }

  return NextResponse.next()
}

export const config = {
  matcher: ['/(?!api|_next/static|_next/image|favicon.ico).*/'],
}
```

BETTER AUTH & SECURITY

Protected Route di Server Component

Defense in depth: cek session di proxy.ts DAN di Server Component. Tidak hanya satu tempat.

```
// Meski sudah ada proxy.ts yang redirect ke login,  
// TETAP cek session di Server Component ini!  
// Ini adalah defense in depth.  
import { auth } from '@lib/auth'  
import { headers } from 'next/headers'  
import { redirect } from 'next/navigation'  
  
export default async function DashboardPage() {  
  // ✅ Selalu verify session di Server Component!  
  // Jangan hanya andalkan proxy.ts  
  const session = await auth.api.getSession({  
    headers: await headers(),  
  })  
  
  if (!session?.user) {  
    redirect('/login') // fallback jika proxy.ts  
    // diypass  
  }  
  
  // session.user.id pasti ada di sini  
  const userData = await db.user.findUnique({  
    where: { id: session.user.id },  
    select: { name: true, email: true, role: true },  
  })  
  
  return (  
    <main>  
      <h1>Dashboard</h1>  
      <p>Halo, {userData?.name}!</p>  
      <p>Role: {userData?.role}</p>  
    </main>  
  )  
}
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Apa perbedaan proxy.ts dan middleware.ts di Next.js 16?

- A) Tidak ada bedanya — hanya rename
- B) proxy.ts berjalan di Node.js runtime. middleware.ts di Edge Runtime (terbatas).
proxy.ts lebih powerful
- C) middleware.ts lebih baru

Q2 — Mengapa perlu cek session di Server Component meskipun proxy.ts sudah redirect?

- A) Tidak perlu — proxy.ts sudah cukup
- B) Defense in depth — proxy bisa dibypass. Server Component adalah actual security layer.
- C) Hanya untuk logging

Q3 — Parameter apa yang WAJIB dipass ke auth.api.getSession() di Server Component?

- A) Tidak perlu parameter
- B) { headers: await headers() } — agar bisa baca cookies dari request
- C) { userId: session.user.id }

Tugas Mandiri

Implementasikan protected routes dengan proxy.ts + Server Component.

01 proxy.ts

Buat proxy.ts di root. Proteksi /dashboard, /admin, /settings. Redirect ke /login jika tidak ada session. Redirect ke /dashboard jika sudah login dan akses /login.

02 Install

bun add @better-fetch/fetch. Ini dibutuhkan untuk betterFetch di proxy.ts. Test: buka /dashboard tanpa login → redirect ke /login.

03 Double Check

Di app/dashboard/page.tsx: tambahkan auth.api.getSession check + redirect. Test: apakah session check di Server Component benar-benar jalan?

04 callbackUrl

Setelah login berhasil, redirect ke callbackUrl (parameter yang di-set oleh proxy.ts saat redirect). User kembali ke halaman yang mereka mau akses.

⚠ proxy.ts adalah first line of defense — bukan satu-satunya! Selalu verify di Server Component juga.

Yang Sudah Kita Pelajari

- ✓ proxy.ts: Node.js runtime, menggantikan middleware.ts yang deprecated di Next.js 16.
- ✓ Fungsi proxy.ts: redirect user belum login ke /login. Redirect logged-in dari auth routes.
- ✓ betterFetch('/api/auth/get-session'): cara baca session di proxy.ts context.
- ✓ Defense in depth: proxy.ts (first line/UX) + Server Component check (actual security).
- ✓ auth.api.getSession({ headers: await headers() }) WAJIB di setiap protected page.

→ Selanjutnya: **Bab 5 — CVE-2025-29927: Jangan Andalkan Middleware Saja**

